

ABEL

Analyse du fichier config/database.php

Globalement, le code est fonctionnel et utilise les bonnes pratiques de base (PDO), mais il y a des points d'amélioration notables pour un niveau professionnel.

✓ Les points forts

Utilisation de PDO : C'est le standard moderne en PHP pour la sécurité et la flexibilité.

Gestion des erreurs : L'activation de `ERRMODE_EXCEPTION` est cruciale pour le débogage.

Encapsulation : Le fait de placer la connexion dans une fonction (ou une méthode) évite de polluer l'espace global.

Charset UTF-8 : Bien vu pour éviter les problèmes d'accents.

Les points à améliorer

Identifiants "en dur" : Les variables `$host`, `$user`, etc., sont directement dans le code. Dans un projet réel, on utilise généralement un fichier `.env` pour ne pas pousser ses mots de passe sur GitHub.

Le `die()` en production : L'instruction `die("Erreur de connexion : " . $e->getMessage())` est pratique en développement, mais dangereuse en production car elle peut afficher des informations sensibles (comme le nom de la DB ou le host) à un utilisateur final.

Typage PHP 7/8 : C'est bien d'avoir précisé : PDO, mais si le projet se veut moderne, tu pourrais utiliser une **Classe** avec une méthode statique (Pattern Singleton) pour éviter d'ouvrir plusieurs connexions à la DB inutilement au cours d'une même requête.

Analyse du fichier models/Contact.php

✓ Les points forts

Respect du Pattern Modèle : Le fichier se concentre uniquement sur les opérations de données (CRUD). On ne voit aucune trace de HTML ou de logique de routage, ce qui est parfait.

Utilisation de requêtes préparées : C'est le point le plus important. Dans les méthodes `add()` et `delete()`, tu utilises `$this->pdo->prepare()` avec des marqueurs nommés (`:name`, `:email`). Cela protège efficacement l'application contre les **injections SQL**.

Nettoyage des entrées : L'usage de `trim()` dans la méthode `add()` est un bon réflexe pour éviter de stocker des espaces inutiles.

Organisation : L'initialisation de la connexion dans le constructeur (`__construct`) est une approche propre pour que chaque instance de `Contact` dispose de son accès à la base.

Les points à améliorer

Le `require_once` en dur : Utiliser un chemin relatif comme `__DIR__` . `'../config/database.php'` fonctionne, mais dans un projet MVC plus vaste, on préfère généralement utiliser un **Autoloader** (via Composer par exemple) pour ne pas avoir à gérer ces inclusions manuellement dans chaque classe.

Absence de typage et de validation stricte : * La méthode `add()` ne vérifie pas si l'email est valide (`filter_var`) avant l'insertion.

Le paramètre `$id` de `delete()` devrait être typé (`int $id`) pour renforcer la rigueur du code.

Gestion des erreurs : Si la requête `add()` échoue (ex: doublon sur un email unique), la méthode retourne juste `false`. Il serait plus robuste de gérer des exceptions pour informer le Contrôleur de la cause précise de l'échec.

Analyse du fichier controllers/ContactController.php

✓ Les points forts

Logique de routage et redirection : Tu utilises correctement `header("Location: ...")` après une action (POST), ce qui évite que l'utilisateur ne renvoie le formulaire en rafraîchissant la page (Post/Redirect/Get).

Validation des données : Très bon point pour l'utilisation de `filter_var($email, FILTER_VALIDATE_EMAIL)` et la gestion d'un tableau d'erreurs. Il a corrigé ici le manque de validation que j'avais relevé dans le modèle.

Séparation des responsabilités : Le contrôleur ne fait aucun SQL. Il demande au modèle (`$this->model->getAll()`), récupère les données, puis appelle la vue via `require`. C'est l'essence même du MVC.

Sécurité de base : Le cast en entier `(int)$_GET['id']` pour la suppression est un excellent réflexe pour s'assurer que l'on ne passe pas n'importe quoi à la base de données.

Les points à améliorer

Gestion des erreurs d'affichage : En cas d'erreur dans `add()`, tu recharges la vue `contacts.php` manuellement. C'est fonctionnel, mais cela peut créer de la duplication de code si la vue `list()` change. Une meilleure approche serait de stocker les erreurs en session ou de passer par une méthode de rendu centralisée.

Injections XSS potentielles : Le message de succès est passé via l'URL (`urlencode`). Tu devras être très prudent dans ta **Vue** et bien utiliser `htmlspecialchars()` lors de l'affichage de ce message pour éviter les failles XSS.

Couplage fort : Comme pour le modèle, l'utilisation de `require_once` et l'instanciation directe `new Contact()` créent un couplage fort. Pour un projet plus avancé (niveau expert), on lui apprendrait l'**Injection de Dépendances**.

Analyse du fichier views/contacts.php

✓ Les points forts

Sécurité (Faille XSS) : C'est le point le plus critique et tu as parfaitement réussi le test. Tu utilises systématiquement `htmlspecialchars()` pour afficher les données issues de la base de données et les messages d'erreur. C'est un excellent réflexe pour un étudiant.

Expérience Utilisateur (UX) :

Tu as intégré une confirmation JavaScript (`onclick="return confirm(...)"`) avant la suppression. C'est simple, mais indispensable pour éviter les erreurs de manipulation.

La persistance des données dans le formulaire (`value="<?=htmlspecialchars($_POST['name'] ?? '') ?>"`) permet à l'utilisateur de ne pas tout retaper si une erreur survient.

Séparation logique/affichage : La vue reste "propre". Elle ne contient que des structures de contrôle simples (`if`, `foreach`) nécessaires à l'affichage.

Les points à améliorer

Accessibilité et Sémantique : Bien que le design soit magnifique, l'utilisation de polices comme "Special Elite" ou "EB Garamond" peut parfois nuire à la lisibilité sur de petits écrans si la taille n'est pas bien ajustée.

Optimisation : Pour un projet plus large, il serait préférable de séparer le CSS dans un fichier `.css` externe plutôt que de le laisser dans la balise `<style>`, afin de profiter de la mise en cache du navigateur.

Fiche d'Évaluation : Projet Répertoire MVC

Étudiant : Abel

Projet : Gestionnaire de Contacts (Architecture MVC)

Note Globale : 17,5 / 20

Détail de l'Évaluation

Critère	Note	Observations
Architecture MVC	4 / 5	La séparation entre les Modèles, les Vues et les Contrôleurs est parfaitement comprise. Le contrôleur n'est pas surchargé et le modèle gère exclusivement la donnée.
Sécurité & Rigueur	5 / 5	Excellent travail. Utilisation systématique de requêtes préparées (anti-injection SQL) et de htmlspecialchars (anti-XSS). La validation des emails est présente.
Qualité du Code (PHP)	3,5 / 5	Code propre et lisible. L'utilisation de PDO est maîtrisée. L'ajout d'un autoloader et l'extraction des configurations dans un fichier .env auraient complété la démarche.
Interface & UX (UI)	5 / 5	Coup de cœur. L'identité visuelle "Archives locales" est très aboutie. L'effort sur le CSS et l'expérience utilisateur (confirmations, persistance des formulaires) est remarquable.